Implementation Document

for

UniPool

Version 1.0

Prepared by

Group #16 **Group Name: Undefined_Behaviour**

Aalok	240011	aalok24@iitk.ac.in
Alankrit Gupta	220110	alankritg22@iitk.ac.in
Anubhav Rana	210162	anubhavr21@iitk.ac.in
Aryan Nigam	220227	naryan22@iitk.ac.in
Bhavishya Gupta	220295	bhavishyap22@iitk.ac.in
Gagandeep Kolambe	220393	gagandeep22@iitk.ac.in
Harshit Gupta	220439	gharshit22@iitk.ac.in
Karan Kumar	230533	karanm23@iitk.ac.in
Naman Mohan Singh	230678	mohans23@iitk.ac.in
Pavi Agarwal	220762	apavi22@iitk.ac.in

Course: CS253

Mentor TA: V Arvind

Date: 27th March 2026

Contents

Contents II

Revisions III

1 Implementation Details 1

1.1 Frontend 1
1.2 Backend 2
1.3 Database 10
1.4 External Services 11
1.5 Build System 11

2 Codebase 12

2.1 Repository 12
2.2 Codebase Navigation 12

3 Completeness 19

3.1 Implemented Features 19
3.2 Future Development Plans 21

Appendix A – Group Log 23

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Aalok Alankrit Gupta Anubhav Rana Aryan Nigam Bhavishya Gupta Gagandeep Kolambe Harshit Gupta Karan Kumar Naman Mohan Singh Pavi Agarwal	Preliminary draft	18/03/25

1 Implementation Details

1.1 Frontend

1.1.1 Programming Language

The frontend is written in Dart, an object-oriented, garbage-collected language inspired by C++, Java, and JavaScript among others. The choice for Dart is influenced by its close integration with the Flutter framework as well as ease of coding, its closeness to mainstream OOP languages, built-in null safety and memory safety, runtime type identification, and the availability of native debugging and performance tools (Dart DevTools).

1.1.2 Framework

The frontend uses the Flutter framework, which greatly simplifies app development with its fast performance and easy-to-understand hierarchy based on OOP principles. Key features include:

- **Widgets:** Flutter considers all UI elements widgets. This allows for uniformity amongst UI elements and a tree-style class hierarchy.
- **Stateful hot reload:** This revolutionary feature alleviates the need for compiling the source code from scratch to test any change in the code, a rather time-consuming process. Changes to UI elements do not change program state. For significant changes, a **stateful hot restart** can be performed.
- **Expressive UI:** Flutter provides a rich set of widgets and colour palettes conforming to the Material Design 3.0 specification with easy customization to suit the app theme, resulting in a beautiful yet familiar UI and an intuitive user experience.
- **Performance:** Flutter provides just-in-time (JIT) and ahead-of-time (AOT) compilation to enhance performance of applications.
- **Portability:** Dart code can, by itself, be compiled to native machine code, JavaScript, or WebAssembly. With the Flutter framework, Dart code can additionally be compiled to suit Android and iOS devices. Moreover, the user interface remains the same across various devices, screen sizes, and operating systems for the same class of devices.

1.1.3 Packages and Libraries

Flutter uses Dart's *pub.dev* package manager and software repository to extend functionality.

- **`flutter`:** Defines widgets corresponding to UI elements and colour palettes as described in the Material Design 3.0 specification.
- **`cloud_firestore`:** Handles cloud storage hosted by Firebase, exposing an API to read, write, and update documents stored.

- **`firebase_messaging`:** Manages real-time messaging between users supported by Firebase's Cloud Firestore service.

- **`one_signal`:** Provides real-time push notification service for mobile apps, integrating with the chat system.

- **`http`:** Sets up an HTTP client to connect and communicate with the backend using the request-response pattern.

- **`provider`:** Aids in state management, allowing efficient and scalable management of application state. It helps in maintaining separation between UI and business logic.

- **`image_picker`:** Used to select images from the device's gallery or capture photos using the camera, making it easy to handle media input in the application.

1.2 Backend

1.2.1 Programming Language

Firebase Platform
UniPool employs a serverless backend architecture powered entirely by Google Firebase. Rather than maintaining a custom application server, the Flutter frontend communicates directly with Firebase services through the official Dart client SDKs. This architecture eliminates server deployment and maintenance overhead while providing production-grade security, scalability, and real-time capabilities out of the box.

1.2.2 Framework

Cloud Firestore & Firebase SDK The backend utilizes the Firebase framework and the Firestore NoSQL engine. This eases backend development with several out-of-the-box features and libraries that abstract away implementation details while ensuring best practices. Features that stand out include:

- **Real-time Data Listeners:** Firebase provides a "Streams" based framework that pushes data updates to the client immediately. This is utilized for the **Real-time Chat** and **Ride Matching** features.
- **NoSQL Document Model:** This allows for a flexible schema where ride details and user profiles can evolve without complex migrations, inheriting the merits of rapid developer experience.
- **Client-Side Integration:** The framework handles data synchronization and local persistence automatically, ensuring the app works seamlessly even with intermittent campus connectivity.

1.2.3 Packages and Libraries

Libraries included with Firebase are combined with Flutter.

- firebase_core (^4.3.0): Initialises the Firebase SDK before any Firebase service is accessed. Called in main() via Firebase.initializeApp().

- firebase_auth (^6.1.3): Provides the complete Firebase Authentication client SDK. Manages email/password sign-in, sign-up, Google OAuth 2.0, email verification, password reset, and session state via the authStateChanges() stream.

- cloud_firestore (^6.1.1): Provides the Firestore NoSQL document database client SDK. Used to store user profiles, ride documents, and per-ride message sub-collections. Exposes real-time snapshot streams used for live ride listing and chat.

- firebase_storage (^13.0.5): Handles binary file upload and download to Firebase Cloud Storage. Used in ProfileScreen to upload and retrieve user profile photographs.

- google_sign_in (^6.2.1): Provides the platform-native Google OAuth 2.0 sign-in flow. The returned tokens are exchanged for Firebase credentials via GoogleAuthProvider.credential().

- image_picker (^1.0.4): Provides access to the device gallery for selecting profile photos using ImageSource.gallery. Supports configurable JPEG quality compression.

- intl (^0.20.2): Provides date and time formatting utilities (e.g. DateFormat.yMMMMd()). Used across multiple screens to display ride departure dates in a human-readable format.

- uuid (^4.2.1): Generates RFC 4122 version 4 universally unique identifiers for programmatic document ID generation.

1.2.4 API Endpoints

The backend exposes a real-time data layer that the frontend communicates with using the Firebase Firestore and Auth protocols. This design enables a complete rewrite of the frontend without affecting the data structure; similarly, security rules and backend logic can be altered without affecting the frontend. The implementation of each is opaque to the other.

The Firebase SDK exposes the following SDK Calls and authentication methods:

1.2.4.1 Authentication

Firebase Authentication
Firebase Authentication serves as the identity and session management layer. It provides a hardened, security-audited authentication service that handles token issuance, refresh, revocation, and session expiry automatically. The application root in main.dart wraps the entire widget tree in a StreamBuilder<User?> listening to FirebaseAuth.instance.authStateChanges(), routing users reactively between AuthScreen and HomeScreen without explicit navigation logic.

Action	`createUserWithEmailAndPassword()`
Method	Firebase Auth SDK
Purpose	Register a new user account. Immediately dispatches an email verification link to the registered address.

Request Data

MIME type: `application/json`

```
{
"email": "student@gmail.com",

"password": "secure_password"
}
```

Response Status

Success:

- `UserCredential` (User created andverification email sent)

Failure:

- **`Email-already-in-use`** **`FirebaseAuthException`**
- **`Invalid-email`** **`FirebaseAuthException`**
- **`Weak-password`** **`FirebaseAuthException`**

URL

`signInWithEmailAndPassword()`

Method

Firebase Auth SDK

Purpose

Authenticate a user and establish a secure session token.

Request Data

MIME type: `application/json`

```
{
"email": "student@iitk.ac.in/@gmail.com"

 "password": "secure_password"
}
```

Response Status

Success:

- **`UserCredential` (User authenticated)**

Failure:

- **`user-not-found`**
- **`wrong-password`**
- **unverified email: User signed out; verification email dispatched.**

Action

`logout()`

Method

Firebase Auth SDK- FirebaseAuth.instance.signOut()

Purpose	Terminate the current user session. The authStateChanges() stream emits null, causing the root StreamBuilder to navigate to AuthScreen.
Request Data	None.
Response Status	Success: ● Session cleared; user redirected to AuthScreen.

Action	`reset_password()`
Method	FIREBASE Auth SDK- FirebaseAuth.instance.sendPasswordResetEmail(email: email)
Purpose	Dispatch a Firebase-generated password reset link to the user's registered email address.

Request Data	MIME type: `application/json`
	`{`
	`Username, if forgotten.`
	`new_password`
	`}`

Response Status	Success: ● Password changed Failure: ● Old/New password is required ● User not logged in ● Email expired or password already changed

1.2.4.2 Ride Handling

The following Firestore SDK operations are used to read and write application data. All operations are performed client-side using the cloud_firestore package:

SDK Call	`` `collection('rides').add()` ``
Method	Firestore WRITE
Purpose	Create a new ride request.

Request Data	MIME type: `application/json`
	`{`
	`"source": "Hall 1", "destination": "IIT Kanpur Metro", "rideDate": "2026-03-20T10:00:00", "leaderId": "UID_123",`

```
"leaderName": "Student Name", "status": "open", "participants": [],
"createdAt": "Timestamp"
}
```

Response Status

Success:
- Ride created successfully

Failure:
- Permission denied

SDK Call

```
collection('rides').doc(id).update(),
FirebaseFirestore.instance.collection('rides').doc(rideId).update({
'status': 'completed'})
```

Method

Firestore UPDATE

Purpose

Join an existing ride or update status/ Update a ride document's status to 'completed'. Also increments ridesCompleted on the leader's user document via FieldValue.increment(1).

Request Data

“”Json

{ "participants": ["UID_123", "UID_456"], "status": "completed" }

Response Status

Success:
- **void`** (Update successful)

Failure:
- **`permission-denied`** (If user is not the leader)

SDK Call

```
FirebaseFirestore.instance.collection('rides').doc(rideId).delete()
```

Method

Firestore DELETE

Purpose

Delete the ride with ride id `id`.

Request Data

None.

Response Status

Success:
- Ride deleted successfully

Failure:
- Permission Denied
- Ride not found

SDK Call

```
collection('rides').where('status', '==', 'open')
```

Method	Firestore LISTEN (GET)
Purpose	Retrieve details of all currently available, open rides for the "Find Ride" screen.
Request Data	None.(Filters applied via Firestore Query).Optional destination filter string from the predefined rideLocations list.
Response Status	Success: ● Array of `DocumentSnapshot` (Ride data in response body as JSON) Failure: ● User not logged in

SDK Call	`collection('rides').where('leaderId', '==', userId)`
Method	Firestore LISTEN (GET)
Purpose	Retrieve all rides that the current user has created and is leading.
Request Data	None.
Response Status	Success: ● Array of `DocumentSnapshot` (Ride data in response body as JSON) Failure: ● Permission denied ● User not logged in

1.2.4.3 Profile management

SDK Call	`FirebaseFirestore.instance.collection('users').doc(uid).set({...}, SetOptions(merge: true))`
Method	Firestore UPDATE
Purpose	Update profile details and trust scores whose id is *id.*

Request Data	MIME type: `application/json`
	
	`{`
	`"name": "User Name", "phone": "9876543210", "email": "user@iitk.ac.in", "trustScore": 4.5, "ridesCompleted": 10`
	`}`

Response Status	Success: ● **'Void'** Profile updated successfully Failure: ● No fields being updated

SDK Call	`collection('users').doc(id).get()`
Method	Firestore GET/LISTEN
Purpose	Retrieve the profile details of the user whose id is `id`.
Request Data	None.
Response Status	Success: ● Profile details in response body as JSON

1.2.4.4 Chat and Messaging

SDK Call	`collection('rides').where('status', isEqualTo: 'open').snapshots()`
Method	Firestore LISTEN (GET)
Purpose	Retrieve the real-time stream of chat messages for a specific ride room.
Request Data	None.
Response Status	Success: ● **' Array of `DocumentSnapshot` (Message data ordered by timestamp)** Failure: ● `permission-denied` (User is not a participant or leader of the ride)

SDK Call	`collection('rides').doc(rideid).collection('messages').add()`
Method	Firestore WRITE (POST)
Purpose	Send a new chat message to the ride's group chat.
Request Data	MIME type: `application/json`

	`{`
	`"text": "I will be waiting at Hall 1 gate.", "senderId": "UID_123",`
	`"senderName": "Student Name", "timestamp": "ServerTimestamp"`
	`}`

Response Status	Success: • **'Void'** (Message created successfully) Failure: • `permission-denied` (User not authorized to send messages in this ride), `invalid-argument` (Message text is empty)

1.3 Database

UniPool uses Cloud Firestore, a fully managed NoSQL document database provided by Google Firebase. Firestore organises data into collections of documents, where each document is a set of key-value fields. This model offers schema flexibility, real-time synchronisation via snapshot listeners, and offline caching without additional configuration.

Firestore was chosen over a traditional relational database (e.g., SQLite or PostgreSQL) for the following reasons:

- **Real-time listeners:** Firestore's .snapshots() stream powers the live ride list (FindRideScreen) and per-ride group chat (ChatScreen) without polling, which is architecturally impractical with a REST/SQL backend without WebSockets.
- **Schema flexibility:** Documents do not require a fixed column set. New fields such as seat capacity or vehicle type can be added to ride documents in future iterations without migration scripts.
- **Direct client access:** The Firestore Flutter SDK allows screens to query data directly, eliminating the need for a dedicated application server tier and the associated deployment complexity.
- **Offline persistence:** The Firestore SDK caches recently accessed data locally, providing a degree of offline functionality without additional implementation effort.

The following collections constitute the UniPool data model:

users

Stores user profile and activity data, keyed by the Firebase Authentication UID.

Field Name	Type	Constraint	Function
uid (Document ID)	String	Primary Key Set by Firebase Auth	The Firebase Authentication UID of the user. Used as the document identifier.

name	String	Optional	Display name set by the user in ProfileScreen.
photoUrl	String	Optional (not yed added)	Public download URL of the user's profile photograph stored in Firebase Storage.
ridesCompleted	Number	Default: 0	Count of rides marked as completed by this user as a leader. Incremented via FieldValue.increment(1) on ride completion. Serves as the activity/trust metric shown on the profile screen.

rides

Stores ride listings posted by authenticated users.

Field Name	Type	Constraint	Function
rideId (Document ID)	String	Primary Key Auto-generated by Firestore	Unique identifier assigned by Firestore on document creation.
source	String	Not Null	Departure location. Selected from the 27-item predefined rideLocations list.
destination	String	Not Null Must differ from source	Arrival location. Selected from the same predefined list. Validated client-side to ensure source ≠ destination.
rideDate	String	Not Null ISO 8601 format	Departure date selected by the user via showDatePicker(). Stored as an ISO 8601 date string.
leaderId	String	Not Null Firebase Auth UID	UID of the user who created the ride. Used to restrict mark-complete and delete operations to the ride leader.
leaderName	String	Optional	Display name of the leader at the time of ride creation, read from the users/ collection.
status	String	Not Null Enum: open | completed	Lifecycle state of the ride. New rides are created with status 'open'. Updated to 'completed' by the leader via the mark-complete action.
participants	Array<String>	Default: []	Array of Firebase Auth UIDs of users who have sent a message in the ride's chat. Updated via FieldValue.arrayUnion() on each message send.

createdAt	Timestamp	Not Null	Server-side creation timestamp assigned via Timestamp.now().

rides/{rideId}/messages

A sub-collection scoped under each ride document. Stores the group chat messages for that ride.

Field Name	Type	Constraint	Function
msgId (Document ID)	String	Primary Key Auto-generated by Firestore	Unique identifier assigned by Firestore on document creation.
text	String	Not Null	The message body entered by the user.
createdAt	Timestamp	Not Null	Timestamp of message creation. Used to sort messages in ascending chronological order in the chat view.
senderId	String	Not Null Firebase Auth UID	UID of the user who sent the message. Used to visually differentiate the current user's messages (right-aligned) from others (left-aligned).
senderEmail	String	Not Null	Email address of the sender. Displayed as the author label above each message in the chat view.

1.4 External Services

1.4.1 Google Sign-In

The google_sign_in package (v6.2.1) provides the platform-native Google OAuth 2.0 sign-in flow on Android and iOS. When the user taps the Google sign-in button in AuthScreen, the GoogleSignIn().signIn() method presents the platform account picker dialog. The returned GoogleSignInAuthentication object exposes an idToken and accessToken, which are used to construct a GoogleAuthProvider.credential(). This credential is passed to FirebaseAuth.instance.signInWithCredential() to establish a Firebase session. On first sign-in, a Firestore user document is created or merged under the user's UID to store their profile data.

1.4.2 Firebase Storage (Paid)

Firebase Storage is used exclusively for storing user profile photographs. When a user selects an image in ProfileScreen, the image_picker package returns a local file path. The file is read as bytes and uploaded to Firebase Storage at the path user_images/{uid}.jpg using a StorageReference.putFile() call with 60% JPEG compression. On upload completion, the getDownloadURL() method returns a public CDN-backed URL, which is then written to the

users/{uid} Firestore document as the photoUrl field. The CDN ensures low-latency image delivery across different geographic regions.

1.5 Build System

The Flutter frontend uses the Gradle build system bundled with Android Studio to automate the compile-build-debug process for Android targets. The pubspec.yaml file serves as the Dart/Flutter dependency manifest. The flutter pub get command resolves and downloads all declared dependencies from pub.dev. For iOS targets, the Xcode build system is used. The following key Flutter CLI commands are used in the development workflow:

Command	Purpose
flutter pub get	Resolves and downloads all dependencies declared in pubspec.yaml from pub.dev.
flutter run	Compiles with JIT and launches on a connected device or emulator, enabling hot reload (r) and hot restart (R).
flutter build apk	Produces a release-mode Android APK with AOT compilation via Gradle. Output placed in build/app/outputs/flutter-apk/.
flutter build ios	Produces a release-mode iOS archive. Requires macOS and Xcode.
flutter analyze	Runs the Dart static analyser, surfacing type errors and lint violations across the entire codebase.
flutter test	Executes widget and unit tests defined in the test/ directory.

2 Codebase

2.1 Repository

The source code for UniPool is hosted on GitHub. The repository can be found at the link:
https://github.com/naman065/UniPool

2.2 Codebase Navigation

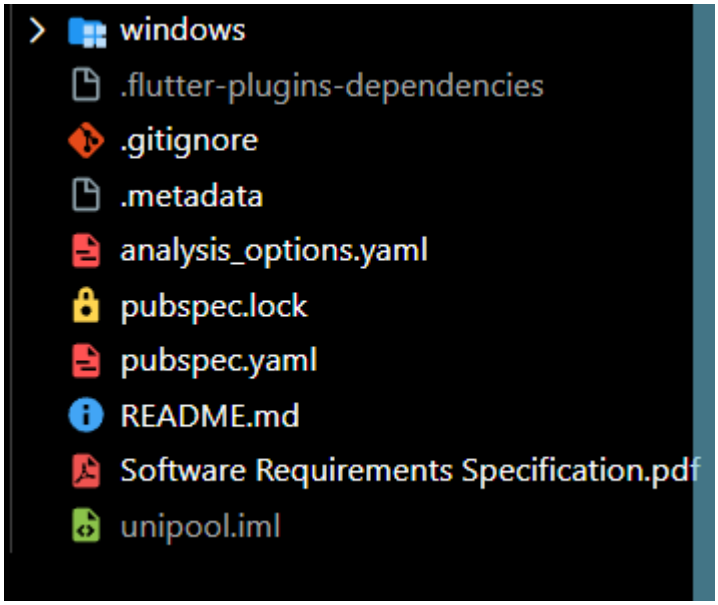
2.2.1 Frontend Directory

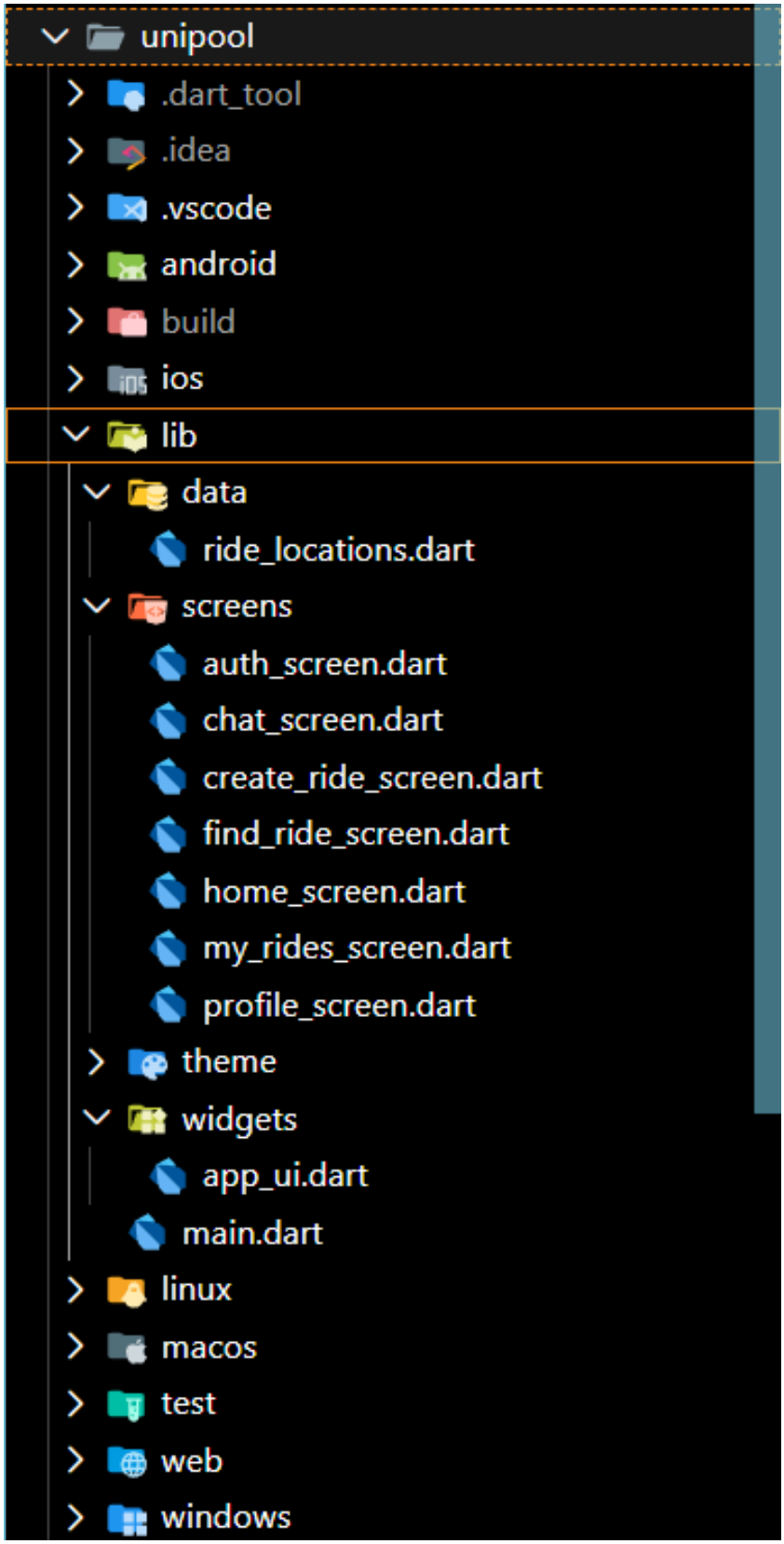
The overall directory structure of the Flutter project is automatically generated by the Flutter CLI for multi-platform targets. The structure below shows only those directories and files that have been created or modified for the purposes of the project. All application source code resides exclusively in the lib/ directory.

The lib/ directory is organised as follows:

Path	Description
lib/main.dart	Application entry point. Initialises Firebase via Firebase.initializeApp(), then mounts UnipoolApp. Contains the root StreamBuilder<User?> listening to FirebaseAuth.instance.authStateChanges() that reactively routes between AuthScreen and HomeScreen.
lib/data/	Static application data layer.
lib/data/ride_locations.dart	Exports the rideLocations constant list — 27 predefined campus and city locations (IIT Kanpur halls, academic zones, Kanpur Metro stations, key city landmarks) used as source/destination options in ride creation and filtering.
lib/screens/	Contains all seven full-screen widgets. Each file is a self-contained StatefulWidget or StatelessWidget that owns its Firestore queries, local state, and UI.
lib/screens/auth_screen.dart	Authentication screen. Handles sign-in, sign-up, Google OAuth, email verification enforcement, and password reset.
lib/screens/home_screen.dart	Post-authentication dashboard. Navigation hub for Create Ride, Find Ride, My Rides, and Profile screens.
lib/screens/create_ride_screen.dart	Ride creation form. Source/destination dropdowns and date picker. Writes to Firestore rides/ collection.
lib/screens/find_ride_screen.dart	Ride discovery screen. Real-time Firestore query of open rides with optional destination filter.

lib/screens/my_rides_screen.dart	Tabbed view of rides led and rides joined. Supports mark-complete and delete actions.
lib/screens/chat_screen.dart	Per-ride real-time group chat. Streams rides/{rideId}/messages sub-collection.
lib/screens/profile_screen.dart	User profile management. Photo upload to Firebase Storage, display name editing, and rides-completed stat.
lib/theme/	Centralised theming.
lib/theme/app_theme.dart	Exports AppColors (dark-mode palette with hex constants and LinearGradient objects) and AppTheme.lightTheme (Material 3 ThemeData factory).
lib/widgets/	Shared reusable UI component library.
lib/widgets/app_ui.dart	Exports all shared widgets: AppGradientBackground, AppPageHeader, AppSurfaceCard, AppPill, AppIconBadge, and the showAppSnackBar utility.




unipool
.dart_tool
.idea
.vscode
android
build
ios
lib
data
ride_locations.dart
screens
auth_screen.dart
chat_screen.dart
create_ride_screen.dart
find_ride_screen.dart
home_screen.dart
my_rides_screen.dart
profile_screen.dart
theme
widgets
app_ui.dart
main.dart
linux
macos
test
web
windows

Screen Details

Authentication Screen — auth_screen.dart

A StatefulWidget managing the complete authentication lifecycle. Boolean state flags (isSignUp, loading, googleLoading, obscure) control UI mode and loading indicators. In sign-in mode, FirebaseAuth.instance.signInWithEmailAndPassword() is called; the email verification status is checked, and unverified users are signed out with a fresh verification link dispatched. In sign-up mode, createUserWithEmailAndPassword() is called, followed immediately by sendEmailVerification(). The Google sign-in chain calls GoogleSignIn().signIn() and exchanges the tokens for a Firebase credential via GoogleAuthProvider.credential(). Error feedback is surfaced through the shared showAppSnackBar() utility.

Home Screen — home_screen.dart

A StatelessWidget serving as the post-authentication navigation hub. Reads FirebaseAuth.instance.currentUser to display a personalised greeting. Provides four navigation tiles: Post a Ride (CreateRideScreen), Find a Ride (FindRideScreen), My Rides (MyRidesScreen), and My Profile (ProfileScreen). Navigation is performed via MaterialPageRoute push calls. The sign-out button calls FirebaseAuth.instance.signOut(), triggering the root authStateChanges() stream to redirect to AuthScreen.

Create Ride Screen — create_ride_screen.dart

A StatefulWidget presenting a ride posting form. Source and destination are selected from DropdownButtonFormField widgets populated with the predefined rideLocations list. The departure date is selected via showDatePicker(). The _submitRide() method validates that all three fields are populated and source ≠ destination before writing to the Firestore rides/ collection with status 'open' and an empty participants array.

Find Ride Screen — find_ride_screen.dart

A StatefulWidget that queries all open rides in real time. A Firestore Query is built against rides/ filtered by status == 'open'. A destination DropdownButtonFormField adds an optional .where('destination', isEqualTo: ...) clause when a specific destination is selected. Results are rendered in a StreamBuilder, providing live updates. Each card shows source, destination, formatted date, and a 'Join Chat' button that navigates to ChatScreen.

My Rides Screen — my_rides_screen.dart

A StatelessWidget with a DefaultTabController (two tabs). The 'I am leading' tab queries rides where leaderId == currentUser.uid; the 'I joined' tab queries rides where participants array-contains currentUser.uid. Each card shows source, destination, date, and a status pill. For leading rides with status 'open', two action buttons are rendered: 'Mark complete' (_completeRide) and 'Delete' (_deleteRide). Mark complete updates status to 'completed' and increments ridesCompleted on the user's profile. Delete shows a confirmation dialog before permanently removing the document.

Chat Screen — chat_screen.dart

A StatefulWidget implementing per-ride real-time group messaging. Accepts rideId and rideDestination as constructor parameters. The message list is driven by a StreamBuilder listening to rides/{rideId}/messages ordered by createdAt ascending. The _sendMessage() method writes a new message document and performs a FieldValue.arrayUnion on the ride's participants field to register the sender. Messages from the current user are rendered right-aligned; others are left-aligned.

Profile Screen — profile_screen.dart

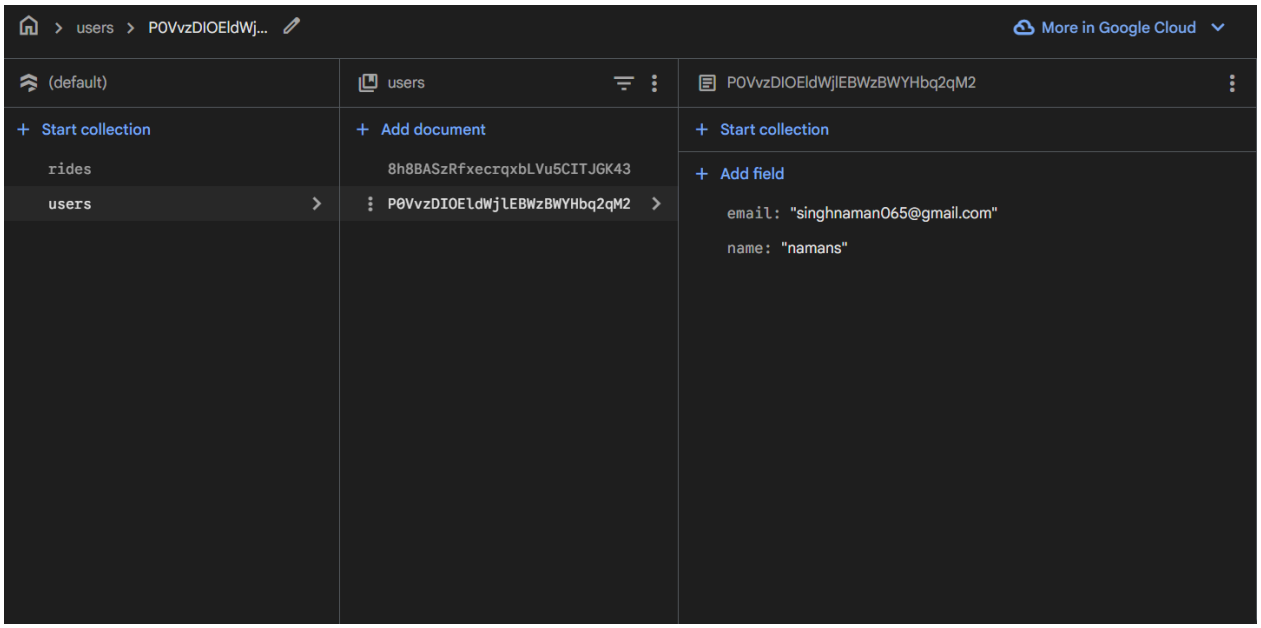
A StatefulWidget that loads and displays the current user's profile. On initialisation, _loadUserData() fetches the users/{uid} Firestore document and populates the display name controller, photo URL, and ridesCompleted count. _pickAndUploadImage() opens the device gallery, uploads the selected image to Firebase Storage at user_images/{uid}.jpg with 60% JPEG compression, retrieves the CDN download URL, and writes it back to Firestore. The display name can be edited and saved to Firestore via _saveProfile().

2.2.2 Backend Directory

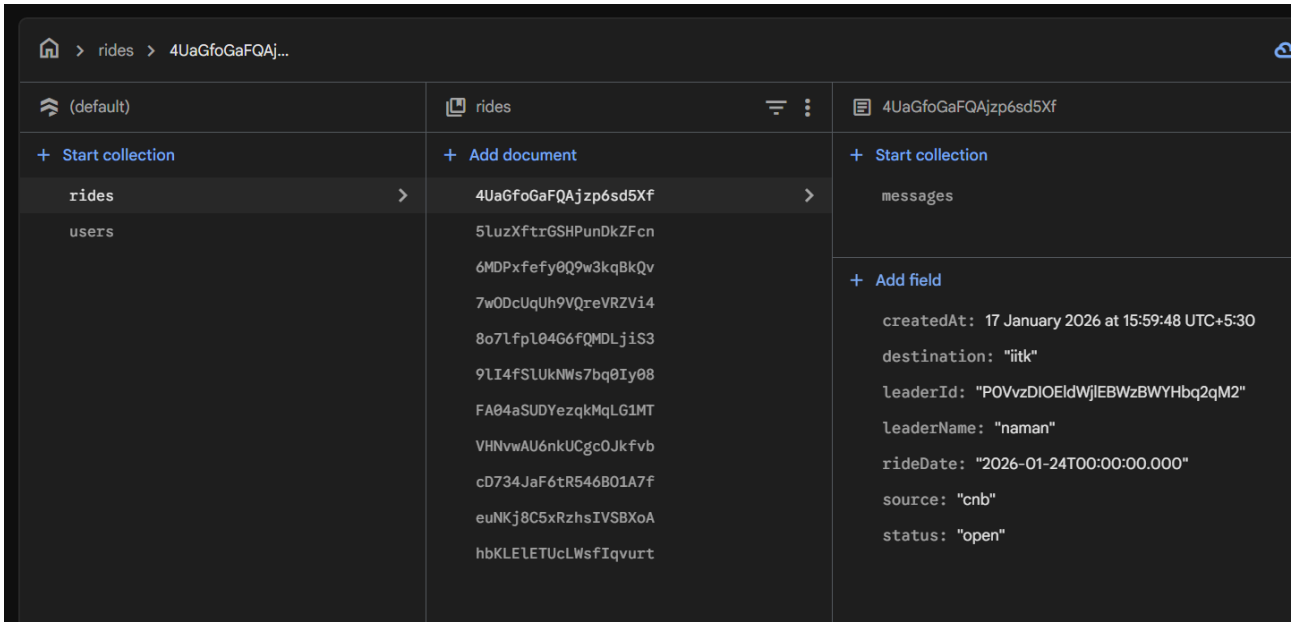
The backend is entirely contained on FIREBASE which is interacted upon using Dart and Firebase Libraries.

2.2.3 Databases

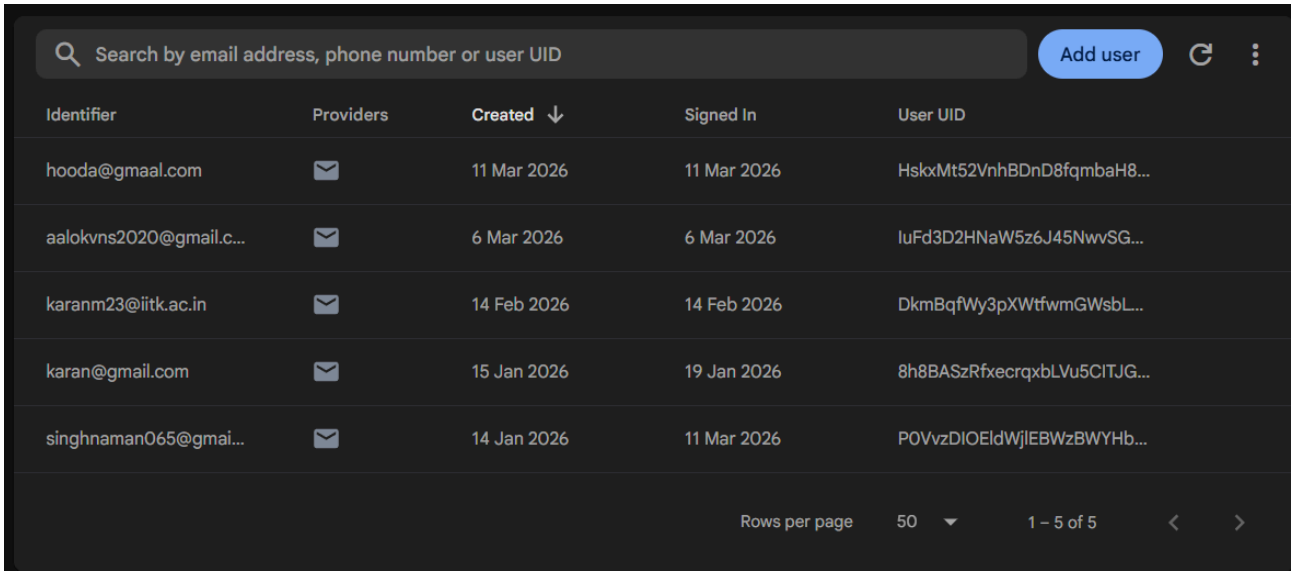
1. **users**



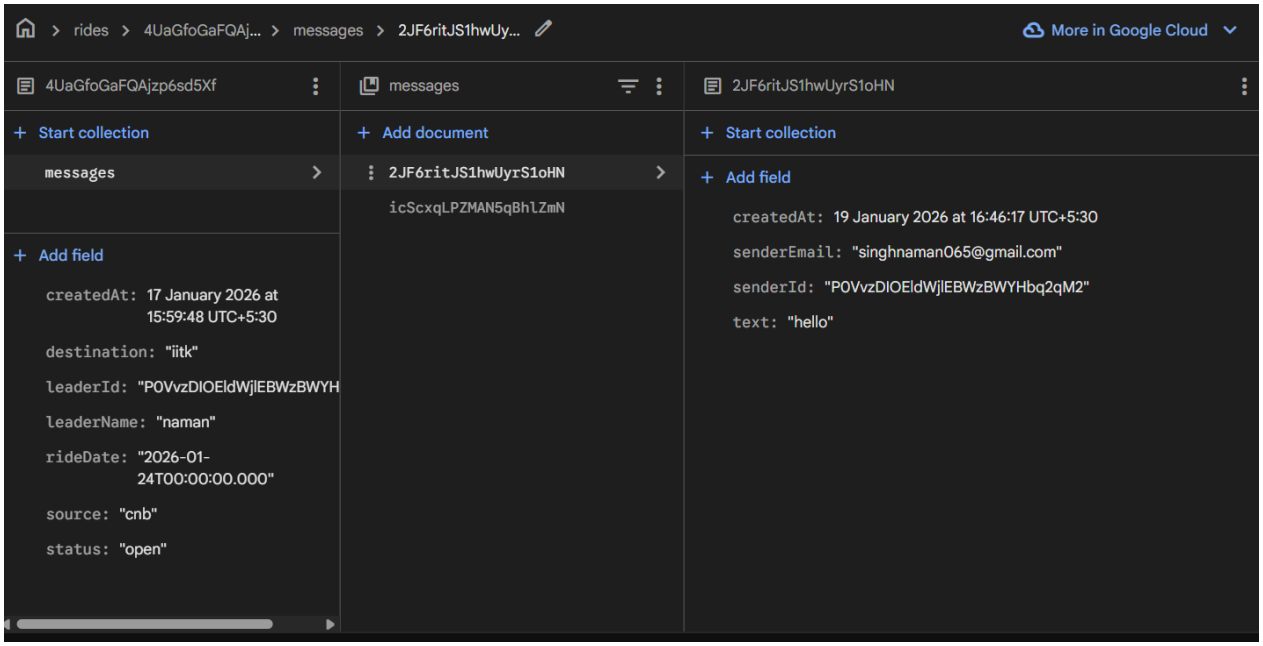
2. **RIdes**



3. Auth



4. Messages



2.2.4 Setting up the Development Environment

2.2.4.1 Setup Steps

To successfully build and run the UniPool application on a local machine, the following prerequisites must be met:

- **Flutter SDK:** Installed and added to the system's PATH.
- **Git:** Installed for version control.
- **IDE:** Visual Studio Code (VS Code) or Android Studio with Flutter and Dart plugins installed.

Step 1: Clone the Repository Open your terminal or command prompt and clone the project repository from GitHub to your local machine:

Bash
1. git clone <https://github.com/naman065/UniPool>
2. cd unipool

Step 2: Install Dependencies Navigate to the project's root directory. Fetch and install all required Flutter and Dart packages specified in the `pubspec.yaml` file by running:

Bash
3. flutter pub get

Step 3: Firebase Configuration *(Note: For security reasons, Firebase configuration files are not committed to version control). (this step can be skipped)*

1. Obtain the `google-services.json` file from the Firebase Console (or from the project administrator).
2. Place this file inside the `android/app/` directory of the cloned repository. This links the local app build to the backend database and authentication services.

Step 4: Device Configuration (Physical Android Device) To test the application on a physical Android device rather than a virtual emulator:

1. Unlock the Android device and go to **Settings > About Phone**.
2. Tap the **Build Number** 7 times to enable **Developer Options**.
3. Navigate back to **Settings > System > Developer Options** and enable **USB Debugging**.
4. Connect the device to the computer via a USB cable. Prompt the device to "Allow USB debugging" if a dialogue appears.

Step 5: Run the Application Ensure your connected device is recognized by the system by running `flutter devices` in the terminal. Once verified, you can launch the app using one of the following methods:

- **Using Visual Studio Code (Recommended):**
    1. Open the project root folder in VS Code.
    2. Select your connected physical device from the bottom-right device selector in the status bar.
    a. Open the terminal and execute:
    Bash- flutter run

- **Using Android Studio:**
    1. Launch Android Studio and select **Open an existing Android Studio project**.
    2. Navigate to the cloned repository and open it.
    3. Select your connected device from the target device drop-down menu in the top toolbar.
    4. Click the green **Run** (Play) button or press `Shift + F10`.

ALTERNATIVE- Directly download the APK file from this path-
\unipool\build\app\outputs\flutter-apk\app-release.apk

Save the apk on your android phone and install it!

3 Completeness

3.1 Implemented Features

3.1.1 Authentication

When the app starts, the authentication screen appears. Users may sign in to an existing account or register a new one from this screen.

- **Sign In —** The user enters their registered email address and password. Upon successful authentication, the user is redirected to the home dashboard. Email verification is enforced; if the account has not been verified, the user is signed out and a fresh verification link is dispatched to their inbox.
- **Create Account —** The user provides an email address and a password of at least six characters. A verification email is dispatched automatically upon successful registration. The user must verify their email before signing in.
- **Forgot Password —** The user enters their registered email address. Firebase dispatches a secure password reset link to that address. Upon tapping the link and setting a new password, the user may sign in normally.

3.1.2 Home Dashboard

Upon successful sign-in, the home dashboard is displayed. It serves as the central navigation hub for the application and contains the following elements:

- A personalised greeting displaying the signed-in user's name derived from their email address.
- **Lead a Ride** action card — navigates to the Ride Creation screen.
- **Find an Open Seat** action card — navigates to the Ride Discovery screen.
- **My Activity** quick-access tile — navigates to the My Rides screen showing rides led and joined.
- **My Profile** quick-access tile — navigates to the Profile Management screen.
- **Sign Out** button — clears the Firebase session and returns the user to the Authentication screen.

3.1.3 Ride Creation

The ride creation screen presents a form where the user, acting as a ride leader, provides the following details:

- **Source location —** selected from a dropdown populated with 27 predefined campus and city destinations (IIT Kanpur halls, academic areas, Kanpur Metro stations, and key Kanpur city landmarks).
- **Destination location —** selected from the same predefined list. The source and destination must be different; an error is shown if they match.
- **Departure date —** selected via the system date picker. Only the current date or future dates may be selected.

Upon submission, the ride is written to the database with a status of open and an empty participants list. The user is returned to the home dashboard with a confirmation message.

3.1.4 Ride Discovery and Filtering

The ride discovery screen displays all currently open rides posted by all users in real time.

- All open rides are shown by default, updating instantly as new rides are posted by other users.
- The user may narrow the list by selecting a specific destination from the destination filter dropdown. Only rides heading to the selected destination are shown.
- Each ride card displays the source location, destination, posting user's name, and formatted departure date.
- Tapping a ride card opens a detail sheet showing the full route, date, and the ride leader's profile including their rides-completed count.
- From the detail sheet, the user may tap **Join and Chat** to enter the ride's dedicated group chat.

3.1.5 Ride Chat

Each ride has a dedicated group chat screen accessible to any user who taps Join and Chat.

- Messages are displayed in real time, streamed from the database and ordered by time of sending.
- The current user's messages appear right-aligned; messages from other participants appear left-aligned with the sender's name shown as a label.
- Sending a message also registers the sender as a participant in the ride, causing it to appear in their I Joined tab on the My Rides screen.
- The chat input supports multi-line messages and can be submitted via the send button or the keyboard action key.

3.1.6 My Rides

The My Rides screen provides a personalised activity view with two tabs:

- **I Am Leading —** Displays all rides for which the current user is the ride leader. Each card shows the route, departure date, and current status (open or completed). For open rides, two management actions are available:
  - **Mark Complete —** Updates the ride's status to completed and increments the leader's rides-completed counter on their profile.
  - **Delete —** Permanently removes the ride from the database after a confirmation dialog. The ride is immediately removed from all live views.
- **I Joined —** Displays all rides in which the current user appears as a participant, i.e. rides where they have sent at least one chat message. Each card provides an Open Chat button to return to that ride's group chat.

An Open Chat button is available on every card in both tabs, allowing the user to navigate directly into the ride's chat at any time.

3.1.7 Profile Management

The profile screen allows the user to view and update their personal information at any time.

- The user's current display name and email address are shown at the top of the screen.

- **Profile Photo —** The user may tap the avatar to open the device gallery and select a photograph. The selected image is compressed and uploaded to cloud storage; the profile is updated automatically upon upload completion.(Paid Feature of Firebase- not functional fully!)
- **Display Name —** The user may edit their display name. Changes are saved to the database by tapping the Save Changes button.
- **Rides Completed —** A read-only counter showing the total number of rides the user has led and marked as completed. This serves as a trust and activity indicator visible to other users on the ride detail sheet.

3.1.8 Logout

The user can sign out at any time by tapping the sign-out icon on the home dashboard. The Firebase session is cleared and the user is immediately redirected to the authentication screen.

3.2 Future Development Plans

3.2.1 Google Maps API integration

- **Live Route Visualisation —** Integrate the google_maps_flutter package to display the planned route between the selected source and destination directly on a map embedded in the ride card or chat screen.

Real-time Location Sharing — Allow ride leaders to share their live GPS location with participants during the journey, reducing uncertainty about pick-up times.

3.2.2 Community engagement and partnerships

- **Extend to other institutions** – The application can be customised for use by other educational institutions, offices and similar establishments.
- **Gamification and Rewards** – Introduce points or incentives for users who use unipool.

3.2.3 Push Notifications via Firebase Cloud Messaging (FCM)

- **New Message Alerts —** Use the firebase_messaging package and Firebase Cloud Functions to dispatch push notifications to ride participants when a new chat message is received, even when the app is in the background.
- **Ride Match Alerts —** Notify users when a new ride matching a saved destination preference is posted.

3.2.4 In-App Fare Splitting

- **Fare Calculation —** After a ride is marked complete, allow the leader to enter the total fare. The application computes each participant's equal share and displays a payment summary within the ride chat.

Payment Integration — Integrate UPI deep-links or a payment gateway (e.g. Razorpay) to allow participants to settle dues without leaving the app.

3.2.5 Porting to iOS

- **Future Expansion to iOS** – Lostify is Android-only now, but Flutter allows easy transition to iOS.
- **Uniform UI/UX Across Platforms** – Ensure consistency in design and features.
- **Apple Maps Alternative for iOS** – Option to switch between Google Maps and Apple Maps.

3.2.6 Peer-to-Peer Rating System

• **Post-ride Rating —** After a ride is completed, prompt participants to rate each other on a 1–5 star scale.

• **Trust Score —** Compute and store an average rating on each user's Firestore document, augmenting the existing ridesCompleted counter to form a comprehensive trust profile. Cloud Functions would enforce that ratings can only be submitted by verified participants of the completed ride.

Appendix A – Group Log

Date & Time	Venue	Minutes
07/03/26 11:00 pm	4th floor, Rajeev Motwani Building	Discussed implementation work, decided the work and team meet timelines, and distributed work among teammates. Decided the tentative dates for completion of frontend and backend components, as well as the start date for integration of both ends.
09/03/26 3:00 pm	Online	Gathered resources for learning Firebase and Flutter and set up our machines for the development process. Redesigned the logo for the app. Basic layout of the app refined for development.
13/03/26 9:00 pm	Online	Revision of components and distribution of individual pages of the application.
15/03/26 9:00 am	1st floor, Rajeev Motwani Building	Met to compile our work and analyse our progress and plan future development processes. Discussed more about the backend routes and decide the app's routes.
19/03/26 10:00 pm	1st floor, Rajeev Motwani Building	Further progress on frontend/learning routes and other backend-related knowledge. Learned more about the APIs necessary to facilitate backend. Revised plans and timelines for backend implementation.
20/03/26 9:00 pm	1st floor, Rajeev Motwani Building	Finished the frontend. Began working on the backend and integration. Started working on the implementation document. Distributed the backend work amongst teammates.
21/03/26 10:00 am	1st floor, Rajeev Motwani Building	Completed backend and frontend. Started working on the integration process. Finalized the design for the databases needed for the code.
22/03/26 6:00 pm	1st floor, Rajeev Motwani Building	Further discussion on integrating frontend/backend. Work on user manual started. Work on testing document started.
25/03/26 2:00 pm	1st floor, Rajeev Motwani Building	Finalized the Implementation Document. Successfully established important routes of the app. Made progress in integration and uploaded the necessary files on GitHub.